

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 7: Online Food Ordering Application using Vue Router and Pinia

SUBMITTED BY

Name: Vijayaraaghavan K S

Register Number: 3122247001066

Branch: Department of CSE

Semester / Year: V Semester / III Year

Subject Code: ICS1511 Batch: 2024–2029

1. Problem Description

Scenario

An online food ordering app with a Home page, an Order section (with nested Food Menu and Cart pages), and an About page, using **Vue Router** for navigation/nested routing and **Pinia** for shared cart state.

Required Routes

Path	Component
/	Home
/order	Order (parent)
/order/menu	Food Menu (nested child)
/order/cart	Cart (nested child)
/about	About

Technology Stack

- Vue 3 (<script setup>), Vite scaffold
- vue-router@4 for top-level and nested routing
- pinia for centralized cart state (actions + getters)

2. Source Code

File: stores/cart.js (Pinia store)

```
import { defineStore } from 'pinia'

// Pinia store holding the shopping cart state, shared by FoodMenu.vue and
// Cart.vue.
export const useCartStore = defineStore('cart', {
  state: () => ({
    items: [] // each item: { id, name, emoji, price, quantity }
  }),

  getters: {
    totalItems: (state) => state.items.reduce((sum, item) => sum +
      item.quantity, 0),
```

```

    totalAmount: (state) => state.items.reduce((sum, item) => sum +
        item.price * item.quantity, 0)
  },
  actions: {
    // Adds a food item to the cart, or increases quantity if it's already
    // there.
    addToCart(food) {
      const existing = this.items.find((item) => item.id === food.id)
      if (existing) {
        existing.quantity += 1
      } else {
        this.items.push({ ...food, quantity: 1 })
      }
    },

    increaseQuantity(id) {
      const item = this.items.find((item) => item.id === id)
      if (item) item.quantity += 1
    },

    decreaseQuantity(id) {
      const item = this.items.find((item) => item.id === id)
      if (item) {
        item.quantity -= 1
        if (item.quantity <= 0) this.removeFromCart(id)
      }
    },

    removeFromCart(id) {
      this.items = this.items.filter((item) => item.id !== id)
    },

    clearCart() {
      this.items = []
    }
  }
})

```

File: router/index.js

```

import { createRouter, createWebHistory } from 'vue-router'
import Home from '../views/Home.vue'
import Order from '../views/Order.vue'
import FoodMenu from '../views/FoodMenu.vue'
import Cart from '../views/Cart.vue'
import About from '../views/About.vue'

// /order is the parent route; /order/menu and /order/cart are its nested
// children and render inside Order.vue's <RouterView>.
const routes = [
  { path: '/', component: Home },

```

```

{
  path: '/order',
  component: Order,
  children: [
    { path: 'menu', component: FoodMenu },
    { path: 'cart', component: Cart }
  ]
},
{ path: '/about', component: About }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router

```

File: main.js

```

import { createApp } from 'vue'
import { createPinia } from 'pinia'
import './style.css'
import App from './App.vue'
import router from './router'

createApp(App).use(createPinia()).use(router).mount('#app')

```

File: App.vue

```

<template>
  <div id="app-shell">
    <nav class="main-nav">
      <RouterLink to="/">Home</RouterLink>
      <RouterLink to="/order">Order</RouterLink>
      <RouterLink to="/about">About</RouterLink>
    </nav>

    <main>
      <RouterView />
    </main>
  </div>
</template>

```

File: views/Order.vue (parent route, nested <RouterView>)

```
<template>
  <section>
    <h2>Order</h2>
    <!-- Sub-navigation for the nested /order/menu and /order/cart routes -->
    <nav class="sub-nav">
      <RouterLink to="/order/menu">Food Menu</RouterLink>
      <RouterLink to="/order/cart">Cart</RouterLink>
    </nav>

    <!-- Renders FoodMenu.vue or Cart.vue depending on the matched child route -->
    <RouterView />
  </section>
</template>
```

File: views/FoodMenu.vue

```
<script setup>
import { useCartStore } from '../stores/cart'

const cart = useCartStore()

const foods = [
  { id: 1, name: 'Pizza', emoji: '🍕', price: 250 },
  { id: 2, name: 'Burger', emoji: '🍔', price: 150 },
  { id: 3, name: 'Pasta', emoji: '🍝', price: 200 },
  { id: 4, name: 'Coke', emoji: '🥤', price: 80 }
]
</script>

<template>
  <div>
    <h3>Available Food</h3>
    <ul class="menu-list">
      <li v-for="food in foods" :key="food.id">
        <span>{{ food.emoji }} {{ food.name }}</span>
        <span>₹{{ food.price }}</span>
        <button @click="cart.addToCart(food)">Add to Cart</button>
      </li>
    </ul>
  </div>
</template>
```

File: views/Cart.vue

```
<script setup>
import { useCartStore } from '../stores/cart'

const cart = useCartStore()
</script>

<template>
  <div>
    <h3>🛒 Cart ({{ cart.totalItems }} items)</h3>

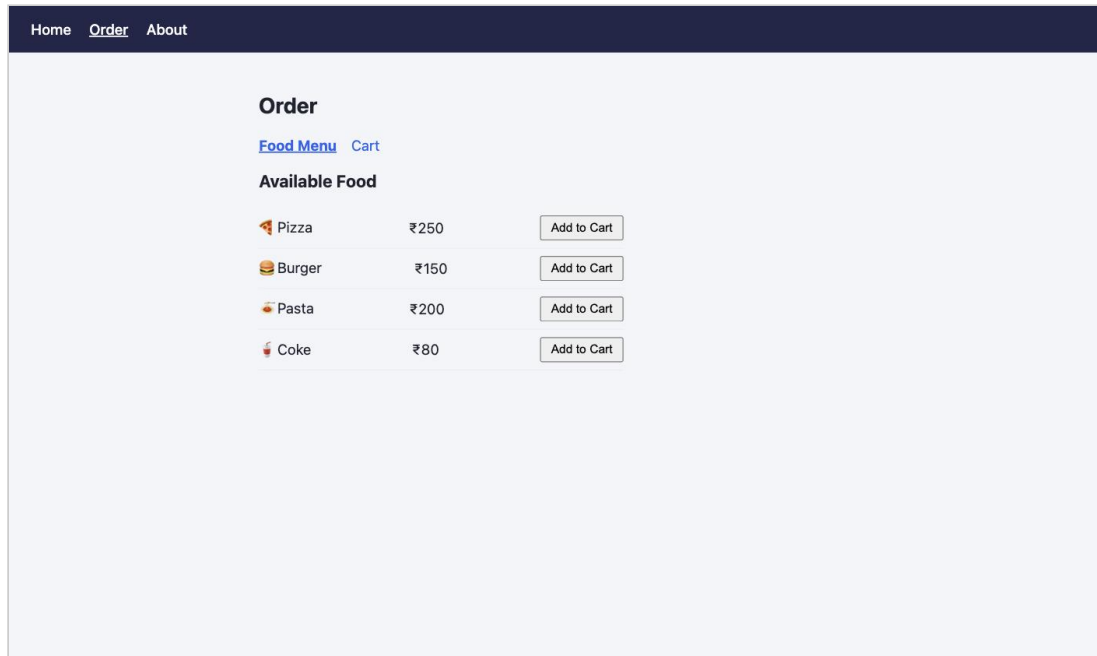
    <p v-if="cart.items.length === 0">Your cart is empty.</p>

    <ul v-else class="cart-list">
      <li v-for="item in cart.items" :key="item.id">
        <span class="name">{{ item.emoji }} {{ item.name }}</span>
        <button @click="cart.decreaseQuantity(item.id)">-</button>
        <span>{{ item.quantity }}</span>
        <button @click="cart.increaseQuantity(item.id)">+</button>
        <span class="line-total">₹{{ item.price * item.quantity }}</span>
        <button @click="cart.removeFromCart(item.id)">Remove</button>
      </li>
    </ul>

    <p v-if="cart.items.length > 0" class="total">Total: ₹
    {{ cart.totalAmount }}</p>
    <button v-if="cart.items.length > 0" @click="cart.clearCart()">Clear
    Cart</button>
  </div>
</template>
```

3. Output Screenshots

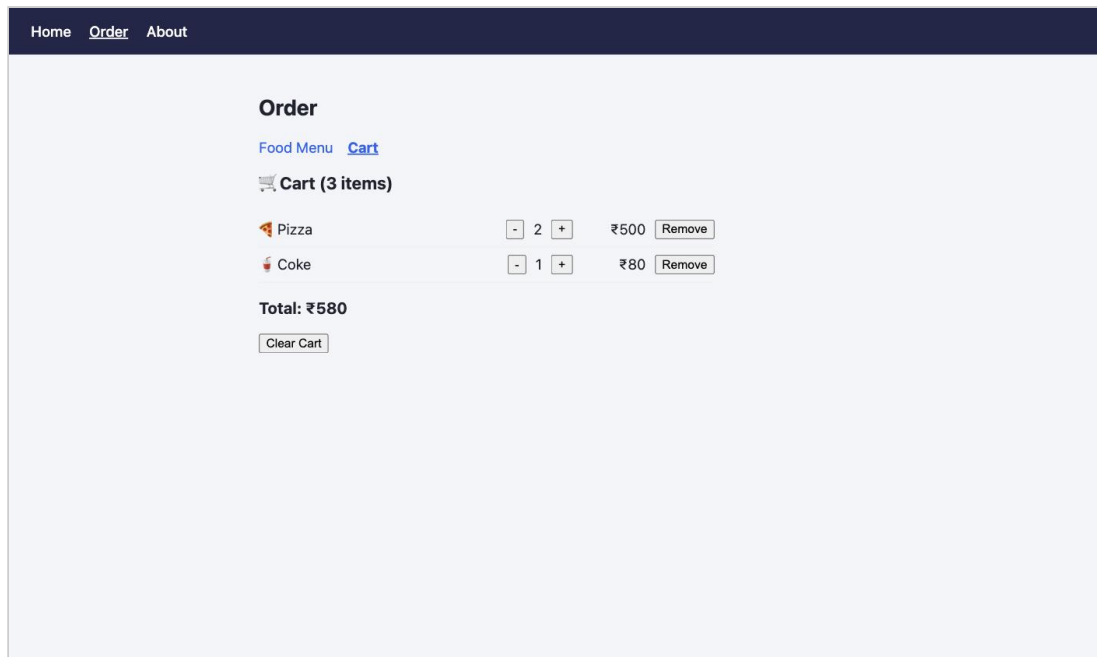
a. Food Menu – /order/menu



Food Menu

Matches the exercise's expected layout exactly: Pizza ₹250, Burger ₹150, Pasta ₹200, Coke ₹80, each with an "Add to Cart" button.

b. Cart – /order/cart



Cart

After adding Pizza twice and Coke once, the Cart page (nested under /order) shows 🍕 Pizza with quantity 2 (₹500) and 🥤 Coke with quantity 1 (₹80), a running **Total: ₹580** computed via the `totalAmount` Pinia getter, and a Clear Cart button — matching the exercise’s sample expected output precisely.

4. Learning Outcomes

This exercise clarified the difference between Vue Router’s top-level routes and **nested routes**: `/order/menu` and `/order/cart` are declared as children of `/order` and render inside `Order.vue`’s own `<RouterView>`, while `/order` itself still renders through `App.vue`’s `<RouterView>`. Pinia made the cart genuinely shared state — `FoodMenu.vue` and `Cart.vue` never talk to each other directly, they both just call `useCartStore()` and the store’s actions (`addToCart`, `increaseQuantity`, `decreaseQuantity`, `removeFromCart`, `clearCart`) and getters (`totalItems`, `totalAmount`) keep everything consistent automatically.